

Manual completo de sintaxis de Python

Basado en todos los apuntes y ejercicios

Índice

1. Introducción	2
2. Variables, tipos de datos y operadores	2
2.1. Tipos básicos y conversión	2
2.2. Operadores aritméticos	2
2.3. Operadores de comparación y lógicos	3
3. Condicionales (if / elif / else)	3
4. Bucles (while y for)	4
4.1. Bucle while	4
4.2. Bucle for	5
5. Funciones	6
5.1. Definición y tipos de funciones	6
5.2. Funciones puras, impuras y mixtas	7
5.3. Llamada a funciones y ámbito	8
6. Listas	9
6.1. Creación y acceso	9
6.2. Métodos de listas	9
6.3. Comprensión de listas	10
7. Diccionarios	10
7.1. Creación y acceso	10
7.2. Métodos de diccionarios	11
8. Cadenas de texto (strings)	12
8.1. Métodos de validación	12
8.2. Métodos de transformación y búsqueda	12
9. Manejo de errores (try/except)	13
10. Módulos y paquetes	14
10.1. Módulo random	14
10.2. Módulo string	14
10.3. Módulo os	15
11. Archivos (lectura y escritura)	15

12. Ejemplos de código útil (recopilación de clases)	16
12.1. Par e impar	16
12.2. Mayor de tres números	16
12.3. Factorial con while	16
12.4. Contar vocales	16
12.5. Número primo	17
12.6. Palabra más larga de una lista	17
12.7. Carrito de compras (lista de diccionarios)	17
12.8. Juego de adivinanza con random	17
12.9. Piedra, papel o tijera	18
12.10. Ahorcado básico	18
13. Errores comunes (depuración)	19
14. Tabla de sintaxis resumida (formulario rápido)	19
15. Conclusión	20

1. Introducción

Este documento recoge de manera exhaustiva la sintaxis de Python, incluyendo todos los temas que aparecen en tus apuntes: variables, condicionales, bucles, listas, diccionarios, funciones, módulos, manejo de errores y mucho más. Cada concepto se presenta en tablas con su estructura y ejemplos prácticos.

2. Variables, tipos de datos y operadores

2.1. Tipos básicos y conversión

Entero (int)	variable = numero	edad = 25
Decimal (float)	precio = 99.90	Decimal (float)
variable = numero_decimal	Texto (str)	variable = "texto"
nombre = "Nahuel"	variable = True/False	es_mayor = True
Booleano (bool)	numero = int("123")	Conversión a int
int(valor)	Conversión a float	float(valor)
decimal = float("3.14")	str(valor)	texto = str(456)
Conversión a str		Tipo de variable
type(variable)	type(edad) # <class 'int'>	

2.2. Operadores aritméticos

blue!60!black			
+	Suma	10 + 5	15
-	Resta	10 - 5	5
*	Multiplicación	10 * 5	50
/	División (decimal)	10 / 3	3.3333
//	División entera	10 // 3	3
%	Módulo (resto)	10 % 3	1
**	Potencia	2 ** 3	8

2.3. Operadores de comparación y lógicos

blue!60!black		
==	igual a	if edad == 18:
!=	distinto de	if edad != 18:
>	mayor que	if edad > 18:
<	menor que	if edad < 18:
>=	mayor o igual que	if edad >= 18:
<=	menor o igual que	if edad <= 18:
and	ambas verdaderas	if edad >= 18 and tiene_dni:
or	al menos una verdadera	if dia == "sábado" or dia == "domingo":
not	invierte el valor	if not lloviendo:

3. Condicionales (if / elif / else)

blue!60!black		
if simple	if condicion:	<pre>1 if numero > 0: 2 print("Positivo")</pre>
if condicion: else:	<pre>1 if numero % 2 == 0: 2 print("Par") 3 else: 4 print("Impar")</pre>	if-else if cond1: elif cond2: else:
	if-elif-else	

blue!60!black		
<pre> 1 if nota >= 9: 2 print(" Sobresaliente ") 3 elif nota >= 7: 4 print(" Notable") 5 elif nota >= 5: 6 print(" Aprobado") 7 else: 8 print(" Desaprobado ") </pre> if anidado val_si_True if cond else val_si_False	<pre> if cond1: if cond2: </pre> <pre> 1 mensaje = "Par" if x % 2 == 0 else " Impar" </pre>	<pre> 1 if edad >= 18: 2 if tiene_dni: 3 print("Puede votar") 4 else: 5 print("Necesita DNI") </pre> Operador ternario

4. Bucles (while y for)

4.1. Bucle while

blue!60!black		
while básico while True: if cond: break	<pre> while condicion: </pre> <pre> 1 while True: 2 n = int(input(" Numero: ")) 3 if n == 0: 4 break </pre> while con continue	<pre> 1 contador = 1 2 while contador <= 10: 3 print(contador) 4 contador += 1 </pre> while con break while cond: if cond2: continue

blue!60!black		
<pre> 1 i = 0 2 while i < 10: 3 i += 1 4 if i % 2 == 5 0: 6 continue 7 print(i) # 8 impares </pre> while con else	<pre> while cond: else: (sin break) </pre>	<pre> 1 i = 0 2 while i < 3: 3 i += 1 4 else: 5 print("Terminó normal ") </pre>

4.2. Bucle for

blue!60!black		
for con range(stop) for i in range(inicio, fin): <pre> 1 for i in range 2 (2, 11, 2): 3 print(i) # 4 2,4,6,8,10 </pre> for con paso negativo for letra in cadena: <pre> 1 for f in frutas: 2 print(f) 3 </pre> for con enumerate	<pre> for i in range(n): </pre> <pre> 1 for i in range(1, 2 11): 3 print(i) # 4 1..10 </pre> for con step <pre> for i in range(inicio, fin, -1): </pre> <pre> 1 for letra in "Python 2 ": 3 print(letra) </pre> for sobre lista <pre> for i, elem in enumerate(lista): </pre>	<pre> 1 for i in range(5): 2 print(i) # 0..4 </pre> for con range(start,stop) <pre> for i in range(inicio, fin, paso): </pre> <pre> 1 for i in range(10, 0, -1) 2 : 3 print(i) # 10..1 </pre> for sobre string for elemento in lista: <pre> 1 for i, fruta in enumerate 2 (frutas): 3 print(i, fruta) </pre> for con else

blue!60!black		
<pre>def nombre(*args):</pre>	<pre>1 def suma_todos(* numeros): 2 return sum(numeros) 3 # suma_todos (1,2,3,4) -> 10 **kwargs (con nombre)</pre>	<pre>def nombre(**kwargs):</pre>
<pre>1 def mostrar(** datos): 2 for k,v in datos. items(): 3 print(k, v) 4 # mostrar(nombre ="Ana", edad =30)</pre>		

5.2. Funciones puras, impuras y mixtas

blue!60!black		
Pura Usa print o modifica variables externas.	Solo return, sin efectos secundarios. <pre>1 def suma_pura(a,b): 2 return a + b</pre> Mixta <pre>1 total = 0 2 def suma_impura(a,b) 3 : 4 global total 5 total = a + b print(total)</pre>	Impura Tiene return y también imprime o modifica algo.

blue!60!black		
<pre> 1 def division(a,b): 2 if b == 0: 3 print(" Error : divisi ón por cero") 4 return None 5 return a / b </pre> Anidada Se llama a sí misma. Debe tener caso base. <pre> 1 cuadrado = lambda x: x **2 </pre>	Definida dentro de otra función. <pre> 1 def factorial(n): 2 if n <= 1: 3 return 1 4 return n * factorial(n-1) </pre> Lambda (anónima)	<pre> 1 def exterior(x): 2 def interior(y): 3 return y * 2 4 return interior(x) + 5 </pre> Recursiva lambda params: expresion

5.3. Llamada a funciones y ámbito

blue!60!black		
Llamada básica global nombre <pre> 1 def prueba(): 2 y = 5 # local </pre>	<pre> resultado = nombre(args) </pre> <pre> 1 x = 10 2 def cambiar(): 3 global x 4 x = 20 </pre> Variable local	<pre> r = suma(5,3) </pre> Variable global Dentro de la función, sin declaración

6. Listas

6.1. Creación y acceso

blue!60!black		
Lista vacía	<code>lista = []</code>	<code>vacía = []</code> Lista con elementos <code>lista[indice]</code>
<code>lista = [e1, e2, ...]</code>	<code>numeros = [1,2,3,4,5]</code>	
Acceder por índice	<code>lista[-1]</code>	<code>ultimo = numeros[-1]</code> Modificar elemento
<code>primer = numeros[0]</code>		
Acceder desde el final		
<code>lista[indice] = nuevo_valor</code>	<code>numeros[2] = 99</code>	<code>len(lista)</code>
<code>len(numeros)</code>	Longitud	
Slicing (rebanada)	<code>lista[inicio:fin:paso]</code>	
		1 <code>numeros[1:3]</code> <i># índices</i> <i>1 y 2</i> 2 <code>numeros[::-1]</code> <i># invertir</i>

6.2. Métodos de listas

	blue!60!black	
	append	lista.append(x)
		<pre> 1 nums = [1,2] 2 nums.append(3) # [1,2,3]</pre>
	lista.insert(i, x)	insert lista.remove(x)
	<pre> 1 nums.insert(1, 99) # [1,99,2,3]</pre>	
	remove	
	lista.pop(i) (último si i no se da)	ultimo = nums.pop() # devuelve 3
	<pre> 1 nums.remove(99) # [1,2,3]</pre>	
	pop	sort
	lista.sort()	lista.reverse()
	<pre> 1 nums.sort() # ascendente 2 nums.sort(reverse= True) # descendente</pre>	
	reverse	

blue!60!black		
<pre> 1 nums.reverse() # invierte el orden clear lista.copy() 1 pos = nums.index (2) # posici ón del 2 count lista.extend(iterable) </pre>	<pre> lista.clear() 1 copia = nums.copy() index lista.count(x) 1 nums.extend([4,5]) # [1,2,3,4,5] </pre>	<pre> 1 nums.clear() # [] copy lista.index(x) 1 cuantos = nums.count(1) extend </pre>

6.3. Comprensión de listas

blue!60!black		
Básica [expr for elem in iterable if cond] <pre> 1 par_impar = [" par" if x %2==0 else " impar" for x in range(5)] </pre> Anidada	<pre> [expr for elem in iterable] 1 2 </pre> <pre> 1 pares = [x for x in range(10) if x % 2 == 0] 2 # [0,2,4,6,8] Con else [expr for a in it1 for b in it2] 1 </pre>	<pre> cuadrados = [x**2 for x in range(5)] 2 # [0,1,4,9,16] Con filtro [expr1 if cond else expr2 for ...] 1 pares = [(x,y) for x in range(3) for y in range(3)] </pre>

7. Diccionarios

7.1. Creación y acceso

blue!60!black		
Vacío <pre>{clave: valor, ...}</pre> 1 nombre = persona["nombre"] Acceso seguro dic[clave] = valor del persona["edad"] Eliminar y devolver dic.popitem() n = len(persona) Verificar existencia	<pre>dic = {}</pre> <pre>persona = {"nombre": "Ana", "edad": 25}</pre> Acceder dic.get(clave, default) persona["ciudad"] = "Madrid" Eliminar clave dic.pop(clave) clave, valor = persona.popitem() Longitud clave in dic	<pre>persona = {}</pre> Con datos dic[clave] edad = persona.get("edad", 0) Agregar/Modificar del dic[clave] edad = persona.pop("edad") Último (3.7+) len(dic) if "nombre" in persona:

7.2. Métodos de diccionarios

blue!60!black		
keys dic.values() 1 2 <pre>for k, v in persona.items(): print(k, v)</pre> update dic.clear()	<pre>dic.keys()</pre> 1 2 <pre>for v in persona.values(): print(v)</pre> items dic.update(otro_dic) persona.clear() Comprensión	<pre>for k in persona.keys(): print(k)</pre> values dic.items() 1 <pre>persona.update({"edad": 26, "profesion": "ingeniero"})</pre> clear {k: v for ...}

blue!60!black		
1	<pre>cuadrados = {x: x**2 for x in range(5)}</pre>	

8. Cadenas de texto (strings)

8.1. Métodos de validación

blue!60!black		
isdigit()	todos son dígitos	"123".isdigit() → True
solo letras	"abc".isalpha() → True	isalpha()
"abc123".isalnum() → True	isalnum()	letras o dígitos
islower()	todo minúsculas	"hola".islower() → True
todo mayúsculas	"HOLA".isupper() → True	isupper()
" ".isspace() → True	isspace()	solo espacios
istitle()	formato título	"Hola Mundo".istitle() → True

8.2. Métodos de transformación y búsqueda

blue!60!black		
lower	cadena.lower()	"HOLA".lower() → "hola"
cadena.upper()	"hola".upper() → "HOLA"	upper
" hola ".strip()	strip	cadena.strip()
→ "hola"	cadena.replace(viejo, nuevo)	"hola mundo".replace("mundo", "python")
replace	"a,b,c".split(",") → ['a','b','c']	split
cadena.split(sep)	join	sep.join(lista)
"-".join(["a","b","c"])	cadena.find(sub)	"Python".find("th") → 2
→ "a-b-c"	count	
find	"ababa".count("ab") → 2	cadena.startswith(prefix)
cadena.count(sub)	startswith	

blue!60!black		
"hola".startswith("hola")	cadena.endswith(suffix)	"archivo.txt".endswith(".txt")
→ True		→ True
endswith		

9. Manejo de errores (try/except)

blue!60!black		
try-except básico	try: ... except: ...	
	<pre> 1 2 3 4 try: n = int(input("Numero : ")) except ValueError: print("No es un nú mero") </pre>	<pre> 1 2 3 4 try: n = int(input("Numero : ")) except ValueError: print("No es un nú mero") </pre>
except NombreError:	<pre> 1 2 3 4 try: resultado = 10 / b except ZeroDivisionError: print("División por cero") </pre>	Error específico <pre> except Error1: except Error2: </pre>
	Múltiples except else: (sin error)	
<pre> 1 2 3 4 5 6 7 try: n = int(input()) print(10/n) except ValueError: print("No es número") except ZeroDivisionError : print(" División por cero") </pre>		<pre> 1 2 3 4 5 6 try: n = int(input()) except ValueError: print("Error") else: print("Número válido: ", n) </pre>
Con else		Con finally

blue!60!black		
finally: (siempre se ejecuta)	<pre> 1 try: 2 archivo = open(" datos.txt") 3 finally: 4 archivo.close()</pre>	

10. Módulos y paquetes

blue!60!black		
Importar módulo	import modulo	import math
from modulo import elem	from math import sqrt, pi	Importar específico import modulo as alias
import random as rnd	Importar con alias from modulo import *	from math import *
Importar todo (no recomendado)		

10.1. Módulo random

blue!60!black		
randint(a,b)	Entero aleatorio entre a y b	random.randint(1,10)
Decimal entre 0 y 1	random.random()	random()
random.choice(["rojo", "verde", "naranja"])	choice(lista)	Elige un elemento al azar
shuffle(lista)	random.shuffle(lista) (in situ)	random.shuffle(cartas)
k elementos sin repetición	random.sample(milista, 3)	sample(lista, k)
random.choices(opciones, k=5)	choices(lista, k)	k elementos con repetición

10.2. Módulo string

blue!60!black		
ascii_letters	a-zA-Z	string.ascii_letters
a-z	string.ascii_lowercase	ascii_lowercase
	ascii_uppercase	A-Z

blue!60!black		
string.ascii_uppercase digits Símbolos de puntuación	string.punctuation	string.digits punctuation

10.3. Módulo os

blue!60!black		
getcwd() Contenido de la carpeta os.chdir("carpeta") makedirs(ruta) Elimina un archivo os.rename("viejo.txt", "nuevo.txt") path.exists(ruta) ¿Es un archivo? os.path.isdir("carpeta") path.getsize(ruta) Une rutas de forma segura absoluta = os.path.abspath("archivo.txt")	Directorio actual archivos = os.listdir() chdir(ruta) Crea carpetas (subcarpetas) os.remove("basura.txt") rename(orig, dest) Verifica si existe os.path.isfile("archivo.txt") path.isdir(ruta) Tamaño en bytes ruta = os.path.join("carpeta", "archivo.txt") path.abspath(ruta)	ruta = os.getcwd() listdir() Cambia de directorio os.makedirs("a/b/c", exist_ok=True) remove(archivo) Renombra archivo/carpeta if os.path.exists("archivo.txt"): path.isfile(ruta) path.isdir(ruta) ¿Es una carpeta? tam = os.path.getsize("archivo.txt") path.join(a,b) Ruta absoluta

11. Archivos (lectura y escritura)

blue!60!black		
Abrir (lectura) open(nombre, "w") f = open("log.txt", "a") Leer todo	open(nombre, "r") f = open("salida.txt", "w") Abrir (añadir) archivo.read()	f = open("datos.txt", "r") Abrir (escritura) open(nombre, "a") contenido = f.read() Leer línea

blue!60!black		
<pre> archivo.readline() lineas = f.readlines() Escribir archivo.close() </pre>	<pre> linea = f.readline() Leer todas las líneas archivo.write(texto) f.close() With (recomendado) </pre>	<pre> archivo.readlines() f.write("Hola mundo\n") Cerrar with open(...) as f: </pre>
<pre> 1 with open("datos .txt", "r") as f: 2 contenido = f.read() </pre>		

12. Ejemplos de código útil (recopilación de clases)

12.1. Par e impar

```

1 if numero % 2 == 0:
2     print("Par")
3 else:
4     print("Impar")

```

12.2. Mayor de tres números

```

1 if a >= b and a >= c:
2     mayor = a
3 elif b >= a and b >= c:
4     mayor = b
5 else:
6     mayor = c

```

12.3. Factorial con while

```

1 def factorial(n):
2     resultado = 1
3     i = 1
4     while i <= n:
5         resultado *= i
6         i += 1
7     return resultado

```

12.4. Contar vocales

```

1 def contar_vocales(cadena):
2     vocales = 0
3     for letra in cadena.lower():
4         if letra in "aeiou":
5             vocales += 1
6     return vocales

```

12.5. Número primo

```

1 def es_primo(n):
2     if n < 2:
3         return False
4     for i in range(2, int(n**0.5)+1):
5         if n % i == 0:
6             return False
7     return True

```

12.6. Palabra más larga de una lista

```

1 def palabra_mas_larga(lista):
2     return max(lista, key=len) if lista else ""

```

12.7. Carrito de compras (lista de diccionarios)

```

1 carrito = []
2 def agregar(nombre, precio, cantidad):
3     carrito.append({"nombre": nombre, "precio": precio, "cantidad":
4                     cantidad})
5
6 def total():
7     return sum(p["precio"] * p["cantidad"] for p in carrito)

```

12.8. Juego de adivinanza con random

```

1 import random
2 secreto = random.randint(1,20)
3 intentos = 3
4 while intentos > 0:
5     n = int(input("Adivina: "))
6     if n == secreto:
7         print("Ganaste")
8         break
9     elif n < secreto:
10        print("Mas alto")
11    else:
12        print("Mas bajo")

```

```

13     intentos -= 1
14 else:
15     print(f"Perdiste , era {secreto}")

```

12.9. Piedra, papel o tijera

```

1 import random
2 opciones = ["piedra","papel","tijera"]
3 cpu = random.choice(opciones)
4 jugador = input("Elige: ").lower()
5 if jugador == cpu:
6     print("Empate")
7 elif (jugador == "piedra" and cpu == "tijera") or \
8      (jugador == "papel" and cpu == "piedra") or \
9      (jugador == "tijera" and cpu == "papel"):
10    print("Ganaste")
11 else:
12    print("Perdiste")

```

12.10. Ahorcado básico

```

1 palabra = "python"
2 adivinadas = []
3 intentos = 6
4 while intentos > 0:
5     estado = ""
6     for letra in palabra:
7         if letra in adivinadas:
8             estado += letra + " "
9         else:
10            estado += "_ "
11    print(estado)
12    letra = input("Letra: ").lower()
13    if letra in adivinadas:
14        print("Ya usaste esa letra")
15        continue
16    if letra in palabra:
17        adivinadas.append(letra)
18        if set(adivinadas) == set(palabra):
19            print("Ganaste")
20            break
21    else:
22        intentos -= 1
23        print(f"Te quedan {intentos} intentos")
24 else:
25    print(f"Perdiste, la palabra era {palabra}")

```

13. Errores comunes (depuración)

blue!60!black		
Falta de dos puntos <pre>if x = 5:</pre> num = <pre>int(input("Numero: "))</pre> Índice fuera de rango <pre>i = 0; while i < 5: print(i)</pre> def suma(a,b): return a+b Concatenar str con int <pre>def f(): x = x + 1</pre>	<pre>if x > 5</pre> <pre>if x == 5:</pre> No convertir input <pre>lista = [1,2,3];</pre> <pre>print(lista[3])</pre> <pre>i = 0; while i < 5:</pre> <pre>print(i); i += 1</pre> Falta return <pre>print("Edad: " + 25)</pre> <pre>def f(): global x; x =</pre> <pre>x + 1</pre>	<pre>if x > 5:</pre> Asignación en comparación <pre>num = input("Numero: ")</pre> <pre>print(lista[2])</pre> Bucle infinito <pre>def suma(a,b): a+b</pre> <pre>print("Edad: " + str(25))</pre> Global sin declarar

14. Tabla de sintaxis resumida (formulario rápido)

blue!60!black		
If <pre>if cond: else:</pre> ver arriba While <pre>for var in</pre> iterable: <pre>def suma(a,b):</pre> return a+b Lista <pre>{clave: valor}</pre> <pre>persona["nombre"]</pre> Try-except <pre>import modulo</pre>	<pre>if cond:</pre> <pre>if x%2==0:</pre> <pre>print("par") else:</pre> <pre>print("impar")</pre> If-elif-else <pre>while cond:</pre> <pre>for i in range(5):</pre> <pre>print(i)</pre> Función <pre>[elem1, elem2]</pre> <pre>persona = {"nombre":</pre> <pre>"Ana"}</pre> Acceso dict <pre>try: except:</pre> <pre>import math</pre>	<pre>if x > 0: print(x)</pre> If-else <pre>if cond1: elif cond2: else:</pre> <pre>while i < 10: i += 1</pre> For <pre>def nombre(params): return</pre> <pre>frutas = ["manzana","pera"]</pre> Diccionario <pre>dic[clave]</pre> ver arriba Importar

15. Conclusión

Esta tabla recopila la sintaxis fundamental de Python que aparece en tus apuntes y ejercicios. Sirve como referencia rápida para estudiar y programar. Recuerda que la práctica constante es la clave para aprender.